





GAMERSDEN: A SMART PLATFORM FOR GAMERS AND GAMING CENTERS

Selva Vignesh C – 24619250

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "GamersDen: A Smart Platform for Gamers and Gaming Centers" is the bonafide work of **SELVA VIGNESH C (24619250)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr.K.Ulaga Priya M.E.,Ph.D
Associate Professor & Head
Of Department AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR

Dr.G.Sangeetha
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the project work and viva-voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is a great delight and a proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARIK. GANESH**, and it is with great pleasure that I take this opportunity to thank him.

I am grateful to our **HEAD OF THE DEPARTMENT, Dr.K.Ulaga Priya**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day.

I extend my warmest thanks to my **INTERNAL GUIDE, Dr.G.Sangeetha**, who has patiently guided me and provided invaluable advice and help. I would like to express my greatest gratitude to our staff members, Librarian and non-teaching staff members of the Computer Centre, who have helped and supported me throughout.

Last but not the least, I wish to thank my parents and my friends for their individual support and interest, who have inspired me and encouraged me to go my own way; without them I would be unable to complete this project.

SELVA VIGNESH C – 24619250

TABLE OF CONTENTS

CHAPTER NO.	TABLE OF CONTENTS	PAGE NO.
	ABSTRACT	**i**
	LIST OF FIGURES	**ii**
	ABBREVIATIONS	**iii**
1	**INTRODUCTION**	**1**
1.1	Overview of the Project	1
1.2	Problem Statement	1
1.3	Objectives of the Project	2
1.4	Scope of the Project	2
1.5	Organization of the Report	2
2	**LITERATURE REVIEW**	**4**
2.1	General	4
2.2	Existing System	4
2.2.1	Literature Survey on Online Reservation Systems	4
2.2.2	Literature Survey on Gaming-Centre Software	5
2.2.3	Technology Survey	5
2.3	Proposed System	5
3	**SYSTEM DESIGN**	**7**
3.1	General	7
3.2	System Architecture	7
3.3	Module Description	8
3.3.1	Authentication and Role Management Module	8
3.3.2	Centre and Console Management Module	8
3.3.3	Session Scheduling Module	8
3.3.4	Discovery and Search Module	8
3.3.5	Booking and Capacity Engine	9
3.3.6	Payment Module	9
3.3.7	Booking Management Module	9
3.3.8	Owner Studio Module	9
3.4	Use Case Model	9
3.5	Booking and Payment Flow	10
3.6	Database Design	11
3.7	API Design	13
3.8	Software Requirements	14
3.9	Design Limitations	14
4	**IMPLEMENTATION**	**15**
4.1	Methodology	15
4.2	Development Environment and Project Structure	15
4.3	Backend Implementation	15

4.3.1	Data Models and Migrations	15
4.3.2	Authentication and Role-Based Access	15
4.3.3	Centre, Console and Session Management	15
4.3.4	Booking and Capacity Engine	16
4.3.5	Payment Processing	16
4.3.6	Cancellation	16
4.4	Frontend Implementation	16
4.4.1	Application Structure and State	16
4.4.2	API Client	17
4.4.3	Responsive Design and Styling	17
4.5	Configuration and Security Measures	17
4.6	Demonstration of the Web Application	17
4.7	Important Code Implementation	20
4.7.1	Capacity Check and Privacy Rules	20
4.7.2	Server-Side Pricing and Payment Hold	21
4.7.3	Role-Based Permission	21
4.7.4	Session Overlap Validation	22
4.7.5	Payment Signature Verification	22
4.7.6	CSRF-Aware API Client	22
4.8	Chapter Summary	22
5	**RESULTS AND DISCUSSION**	**23**
5.1	Introduction	23
5.2	Automated Test Results	23
5.3	Functional Test Results	23
5.4	Observed System Behaviour	24
5.5	Limitations of the Results	24
5.6	Chapter Summary	25
6	**CONCLUSION AND FUTURE ENHANCEMENTS**	**26**
6.1	Conclusion	26
6.2	Future Enhancements	27
	REFERENCES	**28**

ABSTRACT

Finding a PlayStation gaming centre with free player slots usually depends on phone calls and word of mouth, and centre owners find it hard to track slot availability and payments. This project presents GamersDen, a full-stack web application that connects gamers with gaming centres. The backend is built with Django and Django REST Framework and exposes a role-based REST API protected by session authentication and CSRF checks. The frontend is a React single-page application built with Vite.

Customers can browse centres, search by centre name, area or game, view live sessions with the number of player slots remaining, and book a shared or a private session. Each booking request is processed inside a database transaction that checks the remaining capacity, applies a ten-minute payment hold and prevents overbooking. Payments are created through Razorpay and a booking is confirmed only after server-side HMAC signature verification. Owners use a dedicated studio to register consoles with supported games and pricing, publish time-slot sessions with overlap checks and view bookings.

Data is stored in SQLite by default, with MySQL available through configuration. Automated tests for capacity, privacy and role protection passed, and the application was demonstrated with seeded centres, sessions and bookings. Distance-based search, player matchmaking and ratings and reviews are proposed as future work.

Keywords: Gaming Centre Discovery; Online Booking; Slot Capacity Management; Django REST Framework; React; Payment Verification; Full-Stack Web Application.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	System Architecture of GamersDen	7
Figure 3.2	Modules of GamersDen	8
Figure 3.3	Use Case Diagram of GamersDen	10
Figure 3.4	Booking and Payment Flow	11
Figure 3.5	Entity–Relationship Diagram of GamersDen	12
Figure 4.1	Home page after customer login	18
Figure 4.2	Login dialog	18
Figure 4.3	Gaming centres and live sessions on the Discover page	19
Figure 4.4	My bookings page with confirmed and unpaid bookings	19
Figure 4.5	Owner studio with summary cards, Add a console and Publish a session	20
Figure 5.1	Error banner shown when the backend is not reachable	24

ABBREVIATIONS

ABBREVIATION	FULL FORM
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DRF	Django REST Framework
ER	Entity–Relationship
HMAC	Hash-based Message Authentication Code
HTML	Hypertext Markup Language
HTTP(S)	Hypertext Transfer Protocol (Secure)
INR	Indian Rupee
JSON	JavaScript Object Notation
JSX	JavaScript XML (React syntax extension)
ORM	Object-Relational Mapping
REST	Representational State Transfer
SHA	Secure Hash Algorithm
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Console gaming has become a popular social activity, and gaming centres that rent PlayStation consoles by the hour are common in Indian cities. A gamer who wants to play with friends usually has to phone or visit several centres to find out which games are installed, whether a console is free and what it costs. Centre owners face the opposite problem: unsold slots go empty, bookings are noted on paper or in chat messages, and payments are collected by hand.

GamersDen is a full-stack web application that connects gamers with gaming centres. Customers can discover centres, search by centre name, area or game, see live sessions together with the number of player slots that remain, and book either a shared session or a private one. Owners register a gaming centre, add consoles with their supported games and pricing, and publish time-slot sessions through a dedicated owner studio.

The backend is written in Python using Django and Django REST Framework [4], [5]. It exposes a role-based REST API [1] that is protected by session authentication and CSRF checks. The frontend is a single-page application built with React and Vite [6], [15]. Data is stored through the Django ORM in SQLite by default, with MySQL available through configuration. Payments are created through Razorpay [7], and a booking is confirmed only after the payment signature has been verified on the server.

The project shows how a booking workflow with shared capacity, time-limited holds and verified payments can be built with open-source web technologies. It also provides a base for later extensions such as location-based search, player matchmaking and ratings.

1.2 PROBLEM STATEMENT

Gamers do not have one place where they can compare gaming centres, see which games are available and check how many player slots are still open. The information is scattered across phone calls, social media pages and word of mouth, so planning a session with friends takes time and often ends in a wasted trip.

Console sessions are naturally shared. A group of two may not fill a four-player console, and the remaining slots would stay idle. A booking system for this setting must therefore track capacity per player rather than per console, allow several groups to join the same session, and still let a customer reserve a session privately.

Online booking also raises technical problems. Two customers may try to take the last slot at the same moment, which can cause overbooking. Unpaid reservations must not block a slot indefinitely, and a payment must be trusted only after it has been verified by the server and not because the browser reports success. Owners, finally, need a simple tool to manage consoles, games, prices and sessions without technical skills.

Therefore, the project aims to develop a web-based platform that lets gamers discover and book player slots at gaming centres, and lets centre owners publish and manage their sessions, while keeping capacity and payments consistent.

1.3 OBJECTIVES OF THE PROJECT

The main objectives of GamersDen are:

1. To design a role-based web application with separate customer and owner accounts.
2. To let owners register a gaming centre and manage consoles, supported games, pricing and session schedules.
3. To let customers discover published centres and search by centre name, area or game.
4. To show live sessions with the game, start time, price and the number of player slots remaining.
5. To support shared and private bookings while preventing overbooking through transactional capacity checks.
6. To hold a pending booking for ten minutes and stop counting it against capacity if payment is not completed.
7. To integrate Razorpay payments with server-side signature verification and webhook handling.
8. To give customers a booking history and owners a dashboard of consoles, sessions, bookings and verified revenue.
9. To verify the critical booking and access-control rules using automated tests.

1.4 SCOPE OF THE PROJECT

The scope of GamersDen covers two kinds of users. A **customer** can register, log in, browse centres and sessions, book shared or private player slots, pay online, view booking history and cancel a booking. A **centre owner** can register together with a gaming centre, add consoles and supported games, set a pricing model, publish sessions and view the bookings made at the centre.

The current system does not provide distance-based location search, maps, ratings and reviews, or player-to-player matchmaking. Each owner account manages one gaming centre. Refunds are not processed automatically: a cancelled booking is flagged for manual review. The system does not control consoles, track play time on site or bill walk-in customers. These items are listed as future enhancements in Chapter 6.

1.5 ORGANIZATION OF THE REPORT

This report is organized into chapters that explain the development of GamersDen.

Chapter 1 – Introduction: Presents the project overview, problem statement, objectives and scope.

Chapter 2 – Literature Review: Discusses existing reservation systems, gaming-centre management software and the technologies used.

Chapter 3 – System Design: Explains the architecture, modules, use cases, booking and payment flow, database design, API design and software requirements.

Chapter 4 – Implementation: Describes the backend, frontend, payment integration, security measures, a demonstration of the application and important code.

Chapter 5 – Results and Discussion: Presents the automated and functional test results, observed behaviour and limitations.

Chapter 6 – Conclusion and Future Enhancements: Summarizes the project, compares the planned and implemented features and lists possible improvements.

CHAPTER 2. LITERATURE REVIEW

2.1 GENERAL

Online reservation systems have been studied and built for many service industries, including hotels, salons, clinics and sports facilities. They replace manual booking with a web application in which customers choose a time, the system checks availability and the owner manages the schedule. A gaming-centre booking platform shares this basic idea but adds two requirements that are less common: capacity that is divided among several groups on one console, and online payment that must be confirmed before a slot is guaranteed.

This chapter reviews representative reservation systems, the software currently available to gaming-centre operators, and the technologies chosen for GamersDen.

2.2 EXISTING SYSTEM

Existing arrangements fall into three groups. The first is manual booking by phone, messaging applications or walk-in, which is flexible but gives no shared view of availability. The second is general web reservation systems built for a single business. The third is management software sold to gaming-centre operators.

2.2.1 LITERATURE SURVEY ON ONLINE RESERVATION SYSTEMS

Leony and Prayitno [8] describe a web-based reservation system for a salon, built with PHP, HTML and CSS using the Waterfall model and verified with black-box testing. They report that the average transaction time fell from about 15 minutes to about 3 minutes, and they recommend automated notifications and digital payment integration as future work. GamersDen includes online payment from the start.

Adams and Tusime [9] designed a hotel reservation system with an HTML and JavaScript front end and a MySQL back end, where guests make reservations while receptionists and administrators view and manage them. The separation between customer-facing and staff-facing functions is also a core design decision in GamersDen, where customers and owners have different accounts and permissions.

Booking is also a classic transaction-processing problem: two concurrent requests for the last available unit must not both succeed. The transaction-processing literature [2] explains atomicity and isolation as the properties that prevent such conflicts, and GamersDen applies them by locking the session record inside a database transaction while capacity is checked. On the security side, broken access control is listed among the most serious web application

risks [3], which motivated the role-checked endpoints and ownership-scoped queries used in the project.

2.2.2 LITERATURE SURVEY ON GAMING-CENTRE SOFTWARE

Several commercial products target gaming-centre operators. SENET [10] is a cloud-based management system for internet cafes, gaming lounges and esports arenas that supports consoles, an online booking portal, prepaid sessions and deposits. GameBiller [11] provides session tracking, automated time-based billing, memberships, prepaid packages, advance bookings, inventory and revenue reporting. HandyCafe [12] offers session management for PCs and consoles, pricing rules, member accounts and reports.

From the public descriptions reviewed, these products are operator-side tools that concentrate on billing, venue control and reporting. Where they offer online booking, it is tied to a single venue rather than to a public page where a gamer can compare many centres and join a shared session. This gap is the starting point for GamersDen.

2.2.3 TECHNOLOGY SURVEY

The REST architectural style [1] separates the client from the server and uses stateless, resource-oriented requests, which suits a single-page frontend talking to a Django backend. Django [4] provides the ORM, authentication, sessions and administration site, while Django REST Framework [5] adds serializers, viewsets and permission classes. React [6] with the Vite build tool [15] is used for the user interface. Razorpay [7] provides payment orders, a hosted checkout and signed callbacks. Entity–relationship modelling [14] was used for the database design and the incremental development process follows common software-engineering practice [13].

2.3 PROPOSED SYSTEM

GamersDen combines gaming-centre discovery with slot-level booking in one web application. Customers use a public discovery page to see published centres and live sessions, with the number of open player slots shown for each session. A booking reserves only the number of players requested, so other groups can join the same session, or it reserves every slot when the customer chooses a private session.

A new booking is held as pending for ten minutes while the customer pays. Payment is created through Razorpay, and the booking becomes confirmed only after the server verifies the payment signature. Owners manage consoles, supported games, pricing and

sessions from an owner studio, and see their bookings and verified revenue. Table 2.1 compares the proposed system with the existing arrangements.

Table 2.1: Comparison of existing arrangements and GamersDen

Feature	Phone / walk-in booking	Venue management software	GamersDen
Compare centres and games across venues	Manual enquiry	Usually limited to one venue's own portal	Public discovery page with search
Live slot availability	Not available	Depends on the product	Remaining player slots per session
Shared sessions with partial capacity	Informal	Not described in the sources reviewed	Book only the slots needed
Private session booking	Informal	Varies	Reserves all player slots
Payment confirmation	Cash at the venue	Offered by some products	Razorpay with server-side signature verification
Owner dashboard	None	Yes	Consoles, sessions, bookings, verified revenue

CHAPTER 3. SYSTEM DESIGN

3.1 GENERAL

System design describes the architecture and major components of GamersDen. The system is a client–server web application in which a React single-page application communicates with a Django REST API over HTTPS using JSON. The server holds all business rules, including capacity checks, pricing and payment verification, so that the browser can be treated as an untrusted client.

3.2 SYSTEM ARCHITECTURE

The architecture of GamersDen consists of the following components, shown in Figure 3.1:

1. **Client layer:** A React and Vite single-page application with views for discovery and search, login and registration, the booking dialog, the customer's bookings and the owner studio. A small API client (api.js) sends requests with the session cookie and attaches the CSRF token to every state-changing request.
2. **Application layer:** A Django project with one app, core. The URL router maps requests to views and viewsets, serializers validate input and format output, permission classes enforce roles, and a services module holds the booking logic (create_booking, active_players and expire_holds).
3. **Data layer:** The Django ORM maps the models to SQLite by default or to MySQL when configured through environment variables.
4. **External payment service:** Razorpay creates payment orders, provides the hosted checkout in the browser and sends signed webhook events to the server.

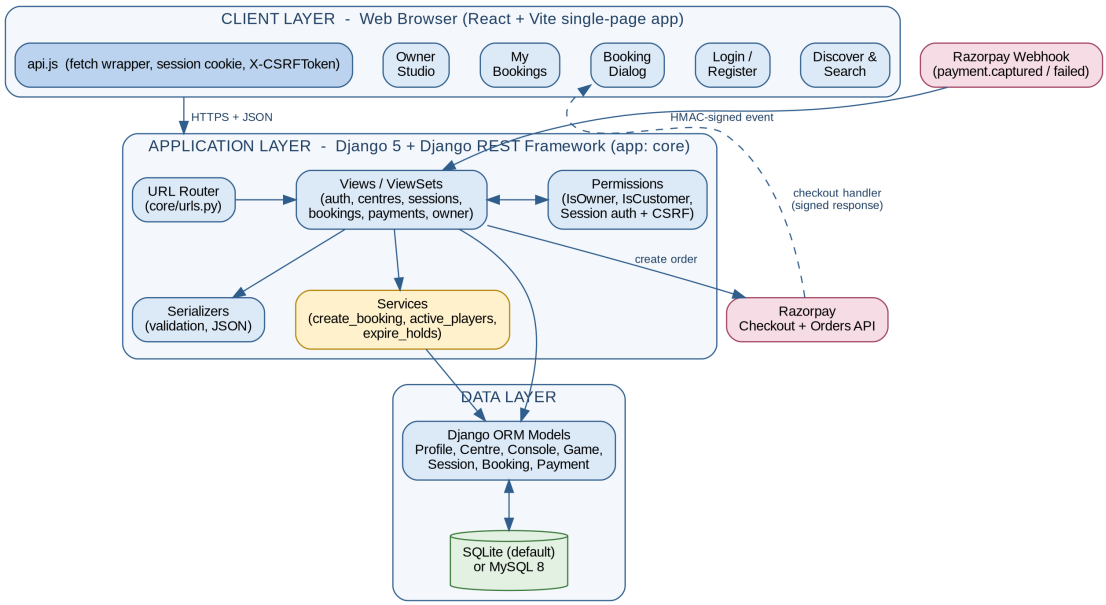


Figure 3.1: System Architecture of GamersDen

3.3 MODULE DESCRIPTION

GamersDen is divided into eight modules to simplify development, testing and maintenance (Figure 3.2).

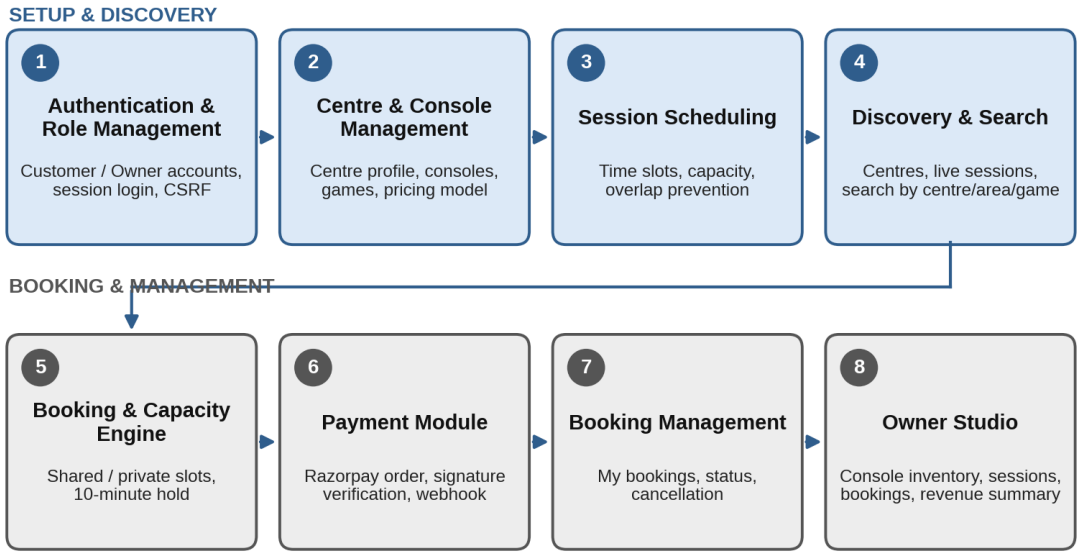


Figure 3.2: Modules of GamersDen

3.3.1 AUTHENTICATION AND ROLE MANAGEMENT MODULE

This module registers users and signs them in. A user registers as a customer or as an owner; an owner must also provide a gaming-centre name and address, from which a centre record is created. The email address is used as the username. Login uses Django session authentication, and the role stored in the user profile decides which endpoints the user may call.

3.3.2 CENTRE AND CONSOLE MANAGEMENT MODULE

Owners maintain the centre profile and the console inventory. Each console has a name, type, maximum number of players, supported games, a pricing model (per player or per session), a price, a default session length and an active flag. Only the owner of a centre can see or change its consoles.

3.3.3 SESSION SCHEDULING MODULE

A session is a time slot on one console for one game. When an owner publishes a session, the server checks that the game is supported by the console, that the end time is after the start time and that the console has no overlapping session. The session capacity is taken from the maximum number of players of the console.

3.3.4 DISCOVERY AND SEARCH MODULE

This public module lists published centres and the available sessions of active consoles. Centres can be searched by name, address or the name of a game played on their consoles, and sessions can be filtered by centre or game. Each session shows the number of booked and available player slots, the price and the pricing model.

3.3.5 BOOKING AND CAPACITY ENGINE

The booking engine is the core of the system. It validates a booking request inside a database transaction, counts the players already holding the session, applies the privacy rules, calculates the amount on the server and creates a pending booking with a ten-minute hold. Only confirmed bookings and unexpired pending holds count against capacity.

3.3.6 PAYMENT MODULE

The payment module creates a Razorpay order for a pending booking, verifies the HMAC-SHA256 signature returned by the checkout, and processes signed webhook events. It keeps the payment state separate from the booking state and records the webhook event identifier so that a repeated event is ignored. A mock mode is available for local development when debugging is enabled.

3.3.7 BOOKING MANAGEMENT MODULE

Customers can list their bookings with the session details and the latest payment status, and can cancel a pending or confirmed booking. Refunds are not claimed automatically; a cancelled booking is reported as requiring manual review.

3.3.8 OWNER STUDIO MODULE

The owner studio shows summary cards for the number of consoles, published sessions and bookings, and the verified revenue. It contains forms to add a console and to publish a session, and lists the console inventory and the bookings made at the centre.

3.4 USE CASE MODEL

GamersDen has two actors. The customer registers or logs in, browses and searches, books slots, pays, and views or cancels bookings. The centre owner logs in, registers the centre, manages consoles and games, publishes sessions and reviews bookings and the dashboard summary. Figure 3.3 shows the use cases.

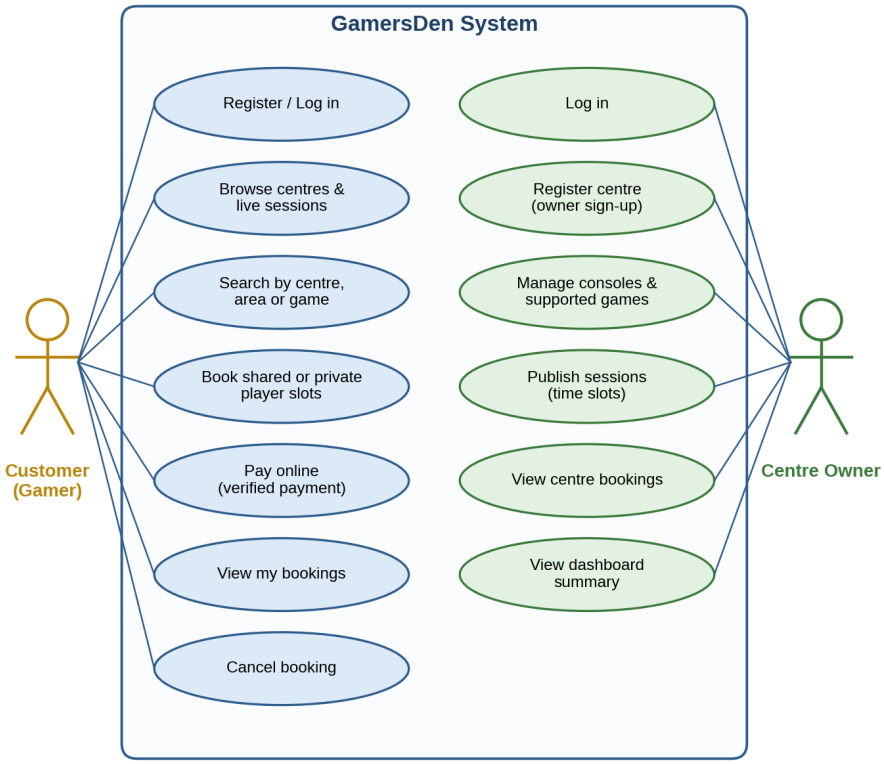


Figure 3.3: Use Case Diagram of GamersDen

3.5 BOOKING AND PAYMENT FLOW

The flow of a booking, from the customer's selection to the confirmed payment, follows these steps (Figure 3.4):

1. The customer selects a session, the number of players and either shared or private mode. The interface shows an estimated total, but the final amount is always recalculated by the server.
2. The browser sends POST /api/booking/create/. The server checks that the user is a customer and that the session is available.
3. Inside a database transaction the server locks the session record, recomputes the players already holding the session and applies the capacity and privacy rules.
4. If the rules pass, a booking is created with status pending and a hold that expires after ten minutes.
5. The browser requests POST /api/payments/order/. The server creates a Razorpay order (or a mock order when debugging is on) and returns the details needed by the checkout.
6. The customer pays in the Razorpay checkout. The browser sends the returned identifiers and signature to POST /api/payments/verify/.
7. The server recomputes the HMAC-SHA256 signature. Only if it matches are the payment marked paid and the booking marked confirmed. A signed webhook gives a second, idempotent confirmation.

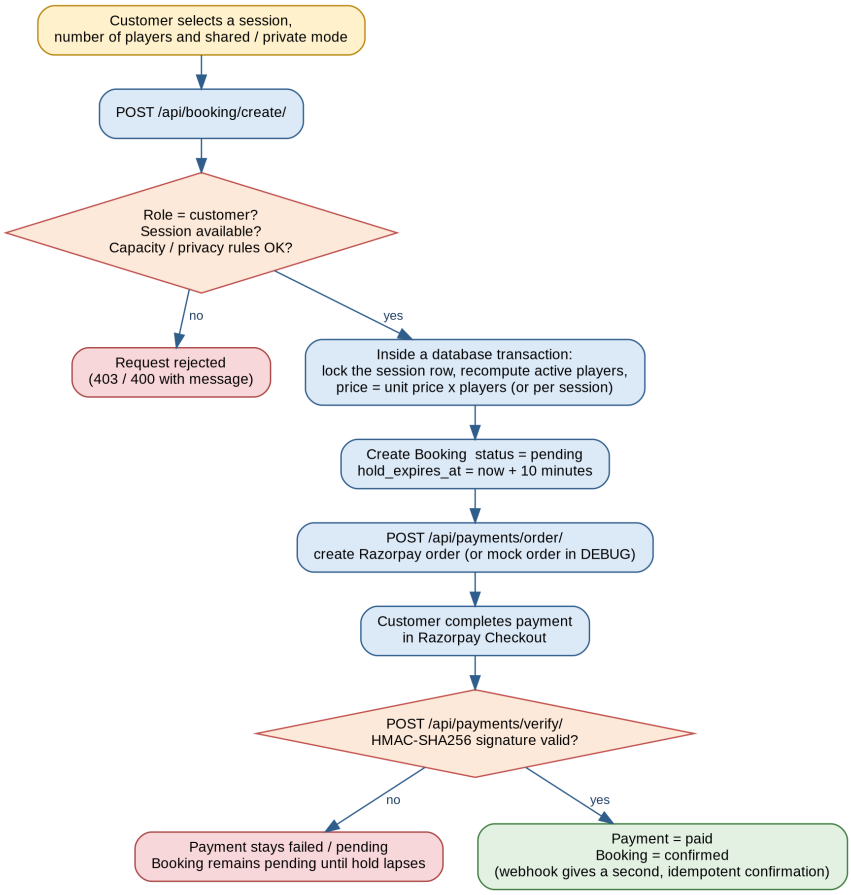


Figure 3.4: Booking and Payment Flow

3.6 DATABASE DESIGN

The database follows the entity–relationship model [14] and is shown in Figure 3.5. Django's built-in User model stores credentials, and the Profile model adds the role and phone number. Each owner has exactly one Centre. A centre has many consoles, a console supports many games through a many-to-many table, and sessions connect a console to a game and a time slot. Customers make bookings against sessions, and each booking can have payment records.

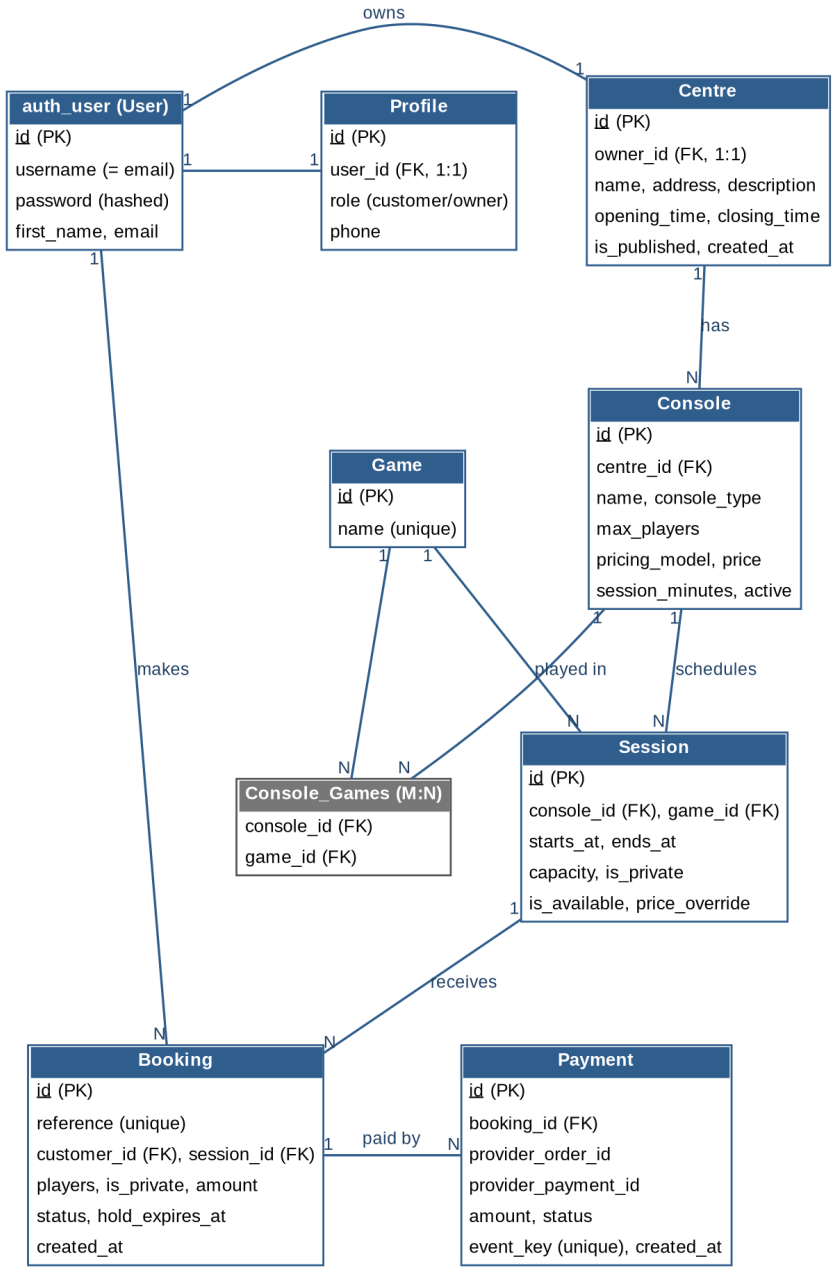


Figure 3.5: Entity–Relationship Diagram of GamersDen

Table 3.1: Main database tables

Table	Purpose	Important design points
auth_user, Profile	Accounts and roles	Email used as username; role is customer or owner; passwords stored hashed
Centre	Gaming centre of an owner	One-to-one with the owner; is_published controls public visibility
Console	Gaming setup in a centre	max_players, pricing_model (per player or per session), price, active flag

Game, Console_Games	Game catalogue and the games a console supports	Game names are unique; many-to-many link table
Session	A bookable time slot	Index on console and time range; optional price_override; capacity and privacy flags
Booking	A customer's reservation	Unique reference; status pending, confirmed, cancelled or expired; hold_expires_at
Payment	Payment attempts for a booking	Provider order and payment ids; unique event_key makes webhook handling idempotent

Bookings and payments use protected foreign keys, so a session or customer cannot be deleted while financial records refer to them.

3.7 API DESIGN

The server exposes a REST API under /api/. Read access to centres, games and sessions is public, while all other endpoints require authentication, and owner endpoints additionally require the owner role. Table 3.2 summarizes the endpoints.

Table 3.2: REST API endpoints of GamersDen

Method	Endpoint	Access	Purpose
GET	/api/csrf/, /api/health/	Public	CSRF token; service health check
POST	/api/auth/register/, /login/, /logout/	Public / user	Create account, sign in, sign out
GET	/api/auth/me/	User	Current user and role
GET	/api/centres/?q=	Public	Published centres, searchable by name, address or game
GET	/api/games/	Public	Game catalogue
GET	/api/sessions/?centre=&game=	Public	Available sessions with slots left and price
POST	/api/booking/create/	Customer	Create a pending booking with a hold
GET	/api/bookings/	User	The user's own bookings
POST	/api/booking/<ref>/cancel/	User	Cancel a pending or confirmed booking
POST	/api/payments/order/	User	Create a payment order for a booking
POST	/api/payments/verify/	User	Verify the signature and confirm the booking
POST	/api/payments/webhook/	Signed	Razorpay webhook (captured / failed)
GET, PATCH	/api/owner/centre/	Owner	Read or update the owner's centre

CRUD	/api/owner/consoles/	Owner	Manage consoles and supported games
CRUD	/api/owner/sessions/	Owner	Publish and manage sessions
GET	/api/owner/bookings/, /owner/summary/	Owner	Centre bookings; dashboard summary

3.8 SOFTWARE REQUIREMENTS

The main software technologies used in the project are listed in Table 3.3.

Table 3.3: Software and libraries

Software or Library	Purpose
Python 3.10+	Backend programming language
Django 5 and Django REST Framework 3.15	Web framework, ORM, authentication and REST API
django-cors-headers, python-dotenv	Cross-origin access for the frontend; environment-based configuration
razorpay (Python SDK)	Creating payment orders
Node.js 18+, Vite, React	Frontend build tooling and user-interface library
lucide-react	Icon set used in the interface
SQLite (default) / MySQL 8 (optional)	Relational database
Modern web browser	Running the single-page application

3.9 DESIGN LIMITATIONS

The design has several limitations. Centres are identified by a text address only, so the system cannot sort by distance. Slot availability is calculated on each request and the interface refreshes it after loading and after a booking, so it is not pushed to other open browsers. Expired holds stop counting against capacity automatically, but the cleanup routine that marks them as expired in the database is not yet scheduled. Row locking in the booking transaction takes full effect on databases that support it, such as MySQL, while SQLite serializes writers at the database level. Finally, each owner can manage only one centre, and refunds need manual handling.

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

GamersDen was developed incrementally. The work moved through requirement analysis, database design, the backend API, the frontend interface, payment integration, testing and a final demonstration. Each increment was run locally and checked in the browser before the next one was added, and the booking rules were fixed early because every other feature depends on them [13].

4.2 DEVELOPMENT ENVIRONMENT AND PROJECT STRUCTURE

The project is split into a backend folder and a frontend folder. The backend is a Django project named config containing one app, core. The frontend is a Vite React project whose source is in the src folder.

```
GamersDen/
  backend/
    manage.py, requirements.txt, .env
    config/   settings.py, urls.py, wsgi.py
    core/     models.py, serializers.py, views.py, services.py,
              permissions.py, urls.py, admin.py, tests.py,
              management/commands/seed_demo.py
  frontend/
    index.html, package.json
    src/      main.jsx, App.jsx, api.js, style.css
```

Project folder structure

4.3 BACKEND IMPLEMENTATION

4.3.1 DATA MODELS AND MIGRATIONS

The core app defines the models Profile, Centre, Game, Console, Session, Booking and Payment. Django migrations create the tables, and a seed_demo management command loads sample centres, consoles, games, sessions and accounts for local development. The Django admin site registers all models for inspection.

4.3.2 AUTHENTICATION AND ROLE-BASED ACCESS

Registration validates the name, email, password (minimum eight characters), role and, for owners, the centre name and address, then creates the user, its profile and the centre in one step. Passwords are stored with Django's password hashing. Login authenticates with the lower-cased email and starts a session. Two permission classes, IsOwner and IsCustomer, read the role from the profile, and the booking endpoint rejects any non-customer account with a 403 response.

4.3.3 CENTRE, CONSOLE AND SESSION MANAGEMENT

Owner endpoints are filtered by the logged-in owner, so one owner can never see another owner's consoles, sessions or bookings. When a console is created the server attaches it to the owner's centre. When a session is published the server verifies that the game is supported by the console, that the end is after the start and that no other session on the same console overlaps, and then sets the capacity from the console.

4.3.4 BOOKING AND CAPACITY ENGINE

The create_booking service runs in a database transaction and locks the session record. It rejects unavailable sessions, invalid player counts, private-session conflicts and requests that exceed the remaining slots. A private booking must reserve all slots and cannot be made once another group has joined. The amount is computed on the server as price multiplied by players for per-player pricing, or the flat price for per-session pricing, and a session may override the console price. The new booking is pending with a hold that ends after ten minutes.

4.3.5 PAYMENT PROCESSING

A payment order can be created only for the customer's own pending booking whose hold has not expired, and never for one that is already paid. In Razorpay mode the server creates an order in rupees converted to paise, using the booking reference as the receipt. The verify endpoint recomputes the HMAC-SHA256 signature of the order and payment identifiers with the secret key and compares it in constant time before marking the payment paid and the booking confirmed inside one transaction. The webhook endpoint checks the signature of the raw request body, ignores an event identifier it has already stored and handles payment.captured and payment.failed events. When the mock mode is selected and debugging is on, the order and verification can be completed locally without a gateway; if credentials are missing in other modes, the server reports that Razorpay is not configured instead of pretending a payment succeeded.

4.3.6 CANCELLATION

A customer can cancel a pending or confirmed booking. The status becomes cancelled and the response states that the refund requires manual review, because gateway refunds need merchant configuration.

4.4 FRONTEND IMPLEMENTATION

4.4.1 APPLICATION STRUCTURE AND STATE

The interface is a single React component tree started from main.jsx. State hooks hold the user, centres, sessions, games, bookings and the form fields, and a view variable switches between three screens: Discover, My bookings and Owner studio. Login and registration open as a modal dialog, and the registration form shows the centre name and address fields only for owners. After sign-in an owner is taken to the Owner studio and a customer to Discover.

The booking dialog lets the customer choose the number of players and a private option that uses all slots, and shows an estimated total with a note that the server recalculates the final amount. After the booking is created the application requests a payment order. In Razorpay mode it loads the checkout script on demand and sends the signed response to the verify endpoint; in mock mode it calls the verify endpoint directly.

4.4.2 API CLIENT

A small wrapper around fetch sends cookies with every request, fetches a CSRF token from the server and adds it as the X-CSRFToken header to every request that is not a GET. Error messages returned by the server are shown to the user in a dismissible banner. The API address comes from an environment variable and falls back to a local default.

4.4.3 RESPONSIVE DESIGN AND STYLING

The interface uses a dark theme written in custom CSS and icons from lucide-react. Breakpoints at 1100, 760 and 420 pixels adapt the layout to tablets and phones. Empty states, such as no centres found or no bookings yet, give the user a clear next step.

4.5 CONFIGURATION AND SECURITY MEASURES

- Session authentication with CSRF protection; session cookies are HTTP-only and SameSite; secure-cookie flags are switched on automatically when debugging is off.
- CORS is limited to the local frontend origins, with credentials allowed, and the same origins are trusted for CSRF.
- Secrets such as the Django secret key, database credentials and Razorpay keys are read from a .env file that is excluded from version control.
- The database is SQLite by default and MySQL when DB_ENGINE is set; the time zone is Asia/Kolkata.

- Every owner query is scoped to the logged-in owner, and customers can read only their own bookings and payments.
- The server, not the browser, calculates prices and decides whether a payment is genuine.

4.6 DEMONSTRATION OF THE WEB APPLICATION

The application was demonstrated with seeded data: two gaming centres, four sessions and nine games. The following screenshots show the main screens (Figures 4.1 to 4.5).

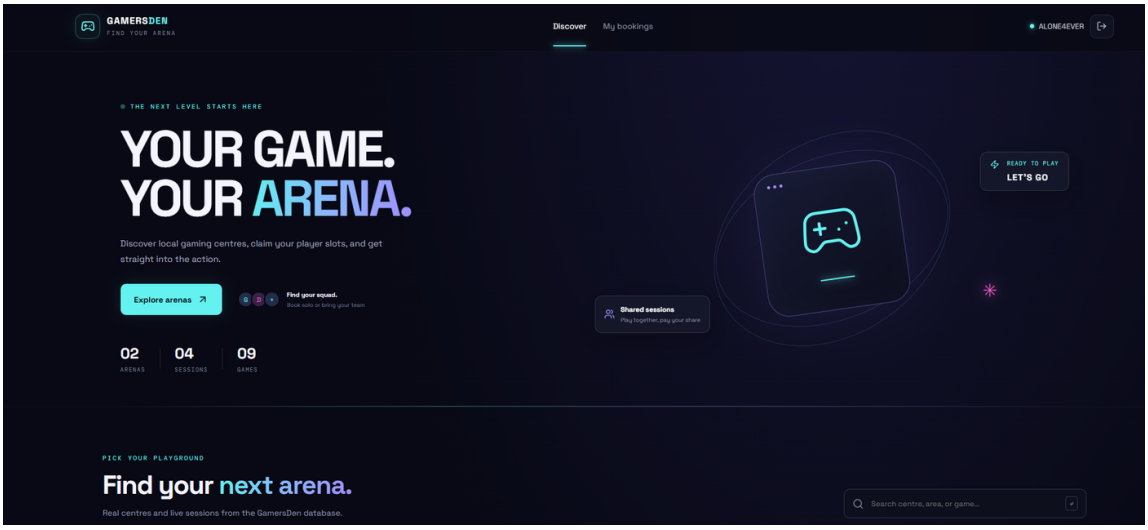


Figure 4.1: Home page after customer login

The home page shows the headline, a call to action and live counters for arenas, sessions and games. After login the navigation bar shows Discover, My bookings and the user name with a logout button.

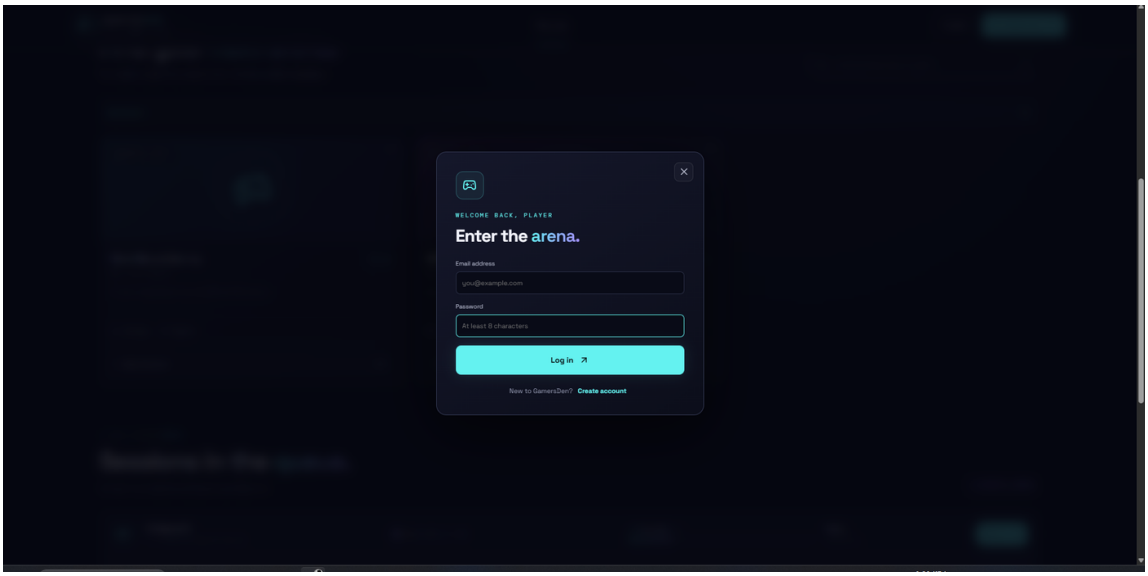


Figure 4.2: Login dialog

Registration offers two account types, Gamer / customer and Gaming centre owner. Choosing the owner type reveals the gaming-centre name and address fields.

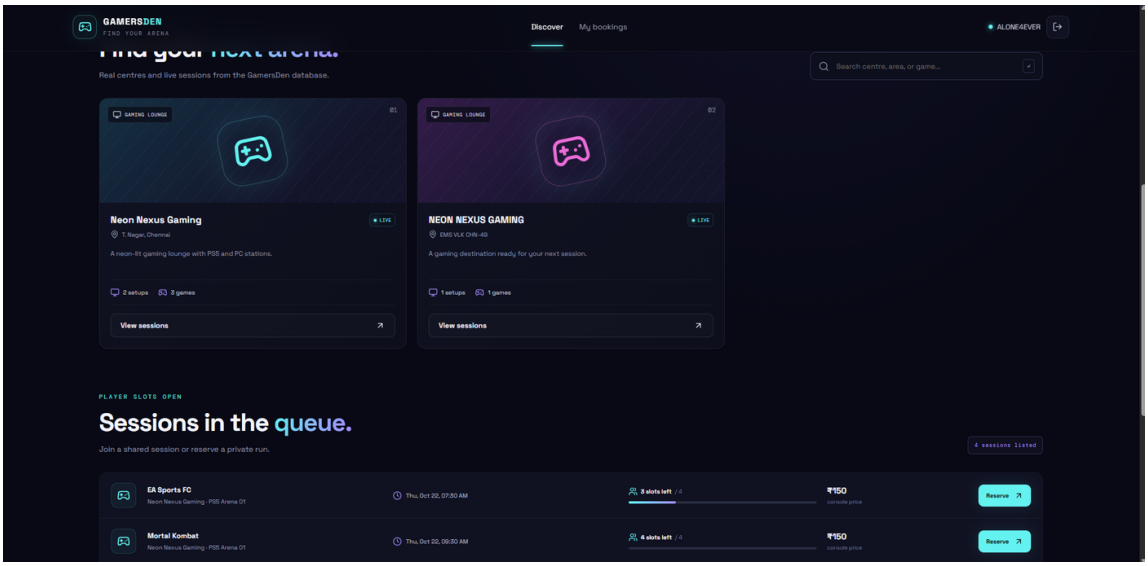


Figure 4.3: Gaming centres and live sessions on the Discover page

The Discover page lists each centre as a card with its location, number of setups and number of games, followed by the sessions in the queue. Each session row shows the game, centre and console, start time, slots left out of capacity, the price and a Reserve button.

The booking dialog asks for the number of players, offers the private-session option and shows the estimated total before the customer pays.

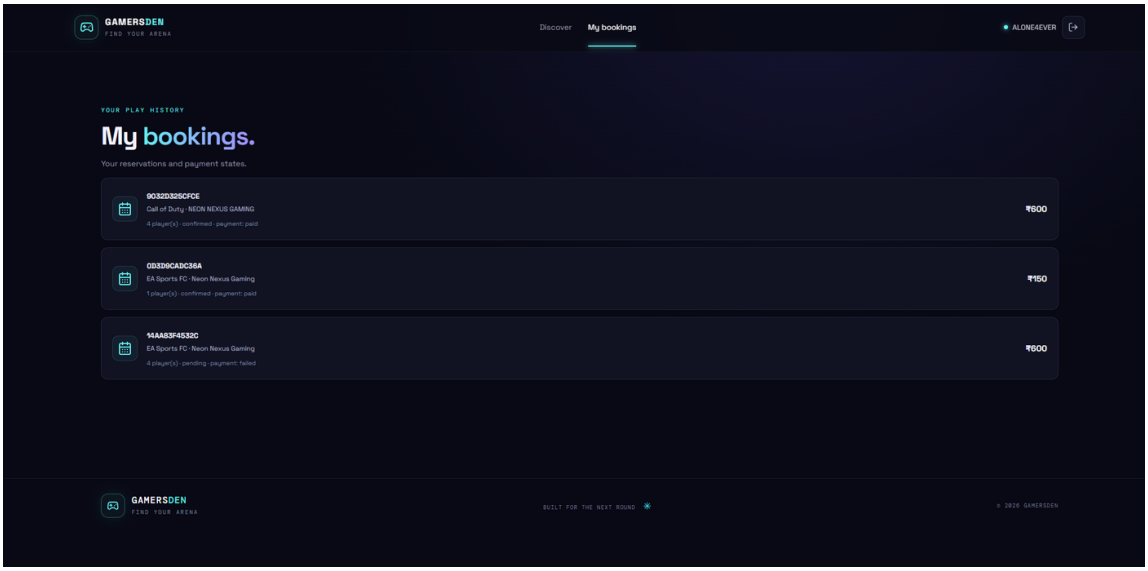


Figure 4.4: My bookings page with confirmed and unpaid bookings

The My bookings page lists each booking with its reference, game and centre, number of players, booking status, payment status and amount. Two bookings are confirmed and paid, while a third is pending because its payment failed.

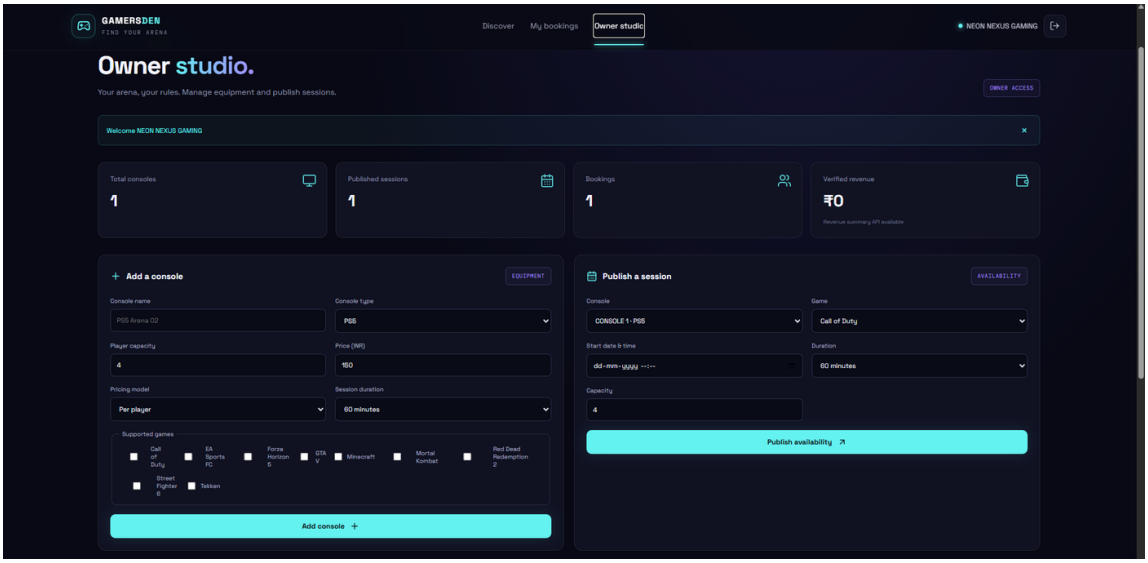


Figure 4.5: Owner studio with summary cards, Add a console and Publish a session

The Owner studio shows summary cards for total consoles, published sessions, bookings and verified revenue. The Add a console form takes the name, type, player capacity, price, pricing model, session duration and supported games, and the Publish a session form takes the console, game, start time, duration and capacity.

4.7 IMPORTANT CODE IMPLEMENTATION

This section presents the important code components. The snippets are taken from the project source.

4.7.1 CAPACITY CHECK AND PRIVACY RULES

Only confirmed bookings and unexpired pending holds count against capacity. The transaction locks the session before the rules are applied.

```
def models_q_active(now):
    from django.db.models import Q
    return (Q(status="confirmed") | Q(status="pending", hold_expires_at__gt=now))


@transaction.atomic
def create_booking(*, customer, session_id, players, is_private=False):
    session =
Session.objects.select_for_update().select_related("console").get(pk=session_id)
    if not session.is_available or not session.console.active:
        raise ValidationError("This session is not available.")
    if players < 1 or players > session.capacity:
        raise ValidationError("Choose a valid number of players.")
    now = timezone.now()
    active = Booking.objects.filter(session=session).filter(models_q_active(now))
    if active.filter(is_private=True).exists() or (session.is_private and
active.exists()):
        raise ValidationError("This private session is already reserved.")
    if is_private and active.exists():
        raise ValidationError("A private session cannot be booked after another
group has joined.")
    if is_private and players != session.capacity:
        raise ValidationError("A private booking must reserve all player slots.")
    if active_players(session) + players > session.capacity:
        raise ValidationError(f"Only {max(0,
session.capacity-active_players(session))} player slots remain.")
```

Capacity and privacy checks in services.py

4.7.2 SERVER-SIDE PRICING AND PAYMENT HOLD

```
console = session.console
unit_price = session.price_override if session.price_override is not None else
console.price
amount = unit_price * players if console.pricing_model == "per_player" else
unit_price
booking = Booking.objects.create(customer=customer, session=session,
players=players, is_private=is_private,
    amount=amount.quantize(Decimal("0.01")),
hold_expires_at=timezone.now()+timedelta(minutes=10))
```

Price calculation and ten-minute hold in services.py

4.7.3 ROLE-BASED PERMISSION

```
def role(user):
    return getattr(getattr(user, "profile", None), "role", None)
class IsOwner(BasePermission):
    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated and
role(request.user)=="owner")
```

Owner permission in permissions.py

4.7.4 SESSION OVERLAP VALIDATION

```
starts=serializer.validated_data["starts_at"];
ends=serializer.validated_data["ends_at"]
if ends <= starts: raise ValidationError("Session end must be after start.")
overlap=Session.objects.filter(console=console, starts_at__lt=ends,
ends_at__gt=starts).exists()
if overlap: raise ValidationError("This console already has an overlapping
session.")
serializer.save(console=console,capacity=console.max_players)
```

Session validation in views.py

4.7.5 PAYMENT SIGNATURE VERIFICATION

```
body=f'{request.data["razorpay_order_id"]}|{request.data["razorpay_payment_id"]}'.
encode()
expected=hmac.new(settings.RAZORPAY_KEY_SECRET.encode(),body,hashlib.sha256).hexdi
gest()
if not hmac.compare_digest(expected,request.data["razorpay_signature"]):
    return Response({"detail":"Invalid payment signature."},status=400)
with transaction.atomic():
    payment.status="paid";
payment.provider_payment_id=request.data["razorpay_payment_id"]; payment.save()
    booking.status="confirmed"; booking.save()
```

Signature check and confirmation in views.py

4.7.6 CSRF-AWARE API CLIENT

```
const res = await fetch(`${API}${path}`, {credentials:'include', ...options,
headers:{
    ...(options.body ? {'Content-Type':'application/json'} : {}),
    ...(csrf && options.method && options.method !== 'GET' ? {'X-CSRFToken':csrf}
: {}),
    ...options.headers
  }});
```

Request wrapper in api.js

4.8 CHAPTER SUMMARY

This chapter described the implementation of GamersDen: the Django models and role-based API, the transactional booking engine, the Razorpay payment flow with signature verification, the React single-page interface and the security measures. Screenshots of the working application and the key code were presented.

CHAPTER 5. RESULTS AND DISCUSSION

5.1 INTRODUCTION

This chapter presents the results obtained from GamersDen. The system was verified in two ways: automated tests for the booking and access-control rules, and functional checks carried out through the web interface with the seeded data.

5.2 AUTOMATED TEST RESULTS

The backend contains three Django test cases that run with the command python manage.py test. All three passed. Together they cover the rules that matter most for correctness: capacity, privacy and access control.

Table 5.1: Automated test cases

Test	What it checks	Result
test_two_groups_can_fill_four_slots	Two groups of two players fill a four-player session, and a further booking of one player is rejected	Pass
test_private_booking_excludes_other_group	After a private booking of all four slots, another customer cannot join the session	Pass
test_role_protected_owner_api	A customer account calling the owner consoles endpoint receives a 403 response	Pass

5.3 FUNCTIONAL TEST RESULTS

Table 5.2 lists the functional checks carried out in the browser. The evidence column refers to the figures in this report.

Table 5.2: Functional test cases

No.	Test case	Expected result	Observed result	Status
1	Open Discover page with the backend running	Centres, sessions and counters are shown	2 arenas, 4 sessions and 9 games shown (Figures 4.1, 4.3)	Pass
2	View session list	Each session shows game, centre, time, slots left and price	EA Sports FC shows 3 slots left of 4 at ₹150; Mortal Kombat shows 4 of 4 (Figure 4.3)	Pass
3	Customer login	Navigation shows My bookings, the user name and logout	Shown after login (Figures 4.1, 4.2)	Pass
4	Book 4 players with successful payment	Booking confirmed, payment paid, amount equals players × price	Call of Duty, 4 players, confirmed and paid, ₹600 (Figure 4.4)	Pass
5	Book 1 player with successful payment	Booking confirmed and paid at the unit price	EA Sports FC, 1 player, confirmed and paid, ₹150 (Figure 4.4)	Pass

6	Payment not completed	Booking is not shown as confirmed	EA Sports FC, 4 players, status pending and payment failed (Figure 4.4)	Pass
7	Owner login	Owner studio with summary cards and the console and session forms	Shown with 1 console, 1 session and 1 booking (Figure 4.5)	Pass
8	Backend not reachable	Error message instead of a crash	Dismissible banner and zero counters (Figure 5.1)	Pass

5.4 OBSERVED SYSTEM BEHAVIOUR

The recorded bookings show that the server calculates the amount correctly: four players at ₹150 per player give ₹600 and one player gives ₹150. The payment state is tracked separately from the booking state, so a booking whose payment did not complete stays pending and is never presented as confirmed. The session list shows the number of slots left out of the capacity, which allows several groups to see and share the same session.

When the backend cannot be reached, the interface keeps working as a page and shows an error banner instead of failing silently, as in Figure 5.1.

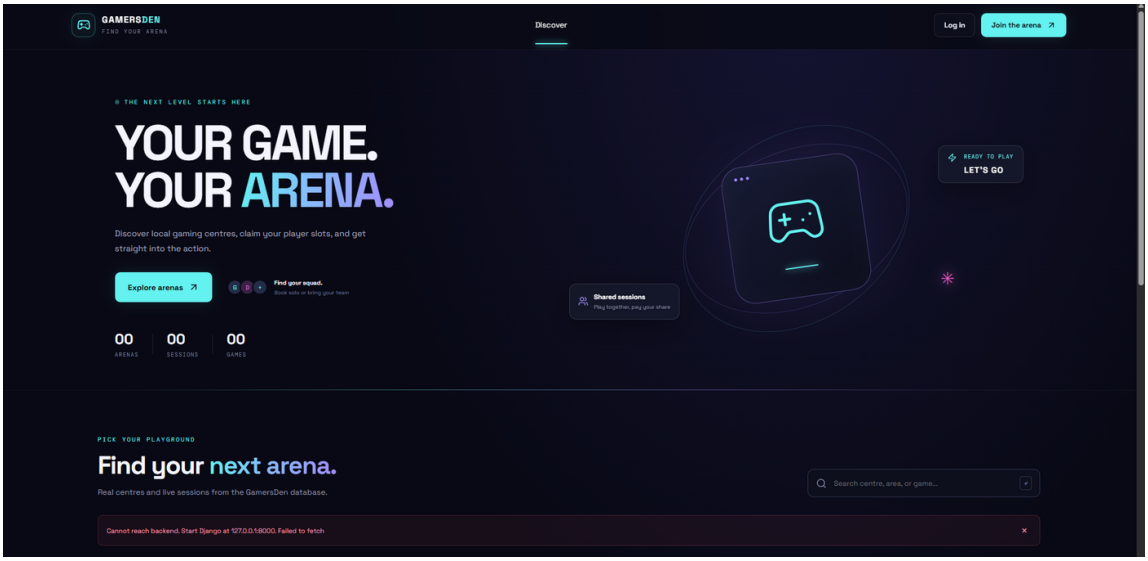


Figure 5.1: Error banner shown when the backend is not reachable

5.5 LIMITATIONS OF THE RESULTS

The testing is limited in scope. The automated suite has three test cases and does not yet cover payment verification, webhooks, cancellation or the owner session rules. The functional checks were done by hand on a small seeded data set and do not measure performance or many users booking at once. The behaviour of row locking under heavy concurrent load on MySQL was not load-tested. Payment verification was exercised in the configured development mode, and a full live-gateway test with real transactions was not part of this project.

5.6 CHAPTER SUMMARY

The automated tests passed, and the functional checks showed correct discovery, booking, pricing, payment status handling and owner management. The results also identified the areas where testing and features can be extended.

CHAPTER 6. CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

GamersDen was developed as a web platform that connects gamers with gaming centres and gives owners a tool to publish and manage sessions. Customers can discover centres, see live slot availability, book shared or private sessions and pay online, and owners can manage consoles, games, prices, sessions and bookings.

The main technical result is a booking engine that checks capacity inside a database transaction, supports shared and private sessions, holds unpaid bookings for ten minutes and confirms a booking only after the payment signature has been verified on the server. Role-based permissions keep customer and owner functions separate. The automated tests for capacity, privacy and access control passed, and the working screens were demonstrated with seeded data.

Table 6.1 compares the features proposed in the project abstract with the final implementation. The core booking and management features are complete, while player matchmaking, ratings and reviews and distance-based search remain for future work.

Table 6.1: Planned features and implementation status

Feature in the abstract	Status	Remarks
Discover gaming centres, view games, prices and slots	Implemented	Centre cards and live session list
Search and filter centres	Implemented	Search by centre name, area or game
Real-time slot availability and online booking	Implemented	Slots left shown per session; shared and private booking
User accounts and profiles	Implemented	Customer and owner roles with login
Owner dashboard for consoles, games, prices and slots	Implemented	Owner studio with summary cards and forms
Online payment	Implemented	Razorpay with signature verification and webhook
Player connection and matchmaking	Future work	Not part of the current build
Ratings and reviews	Future work	Not part of the current build
Location-based discovery	Partly	Text address search; no coordinates or distance sorting
Technology: React, Django REST Framework, database	Implemented with changes	Custom CSS instead of Bootstrap; SQLite default with MySQL option

6.2 FUTURE ENHANCEMENTS

GamersDen can be improved in several ways:

1. **Player matchmaking:** Let gamers find others who want to play the same game and organize multiplayer sessions.
2. **Ratings and reviews:** Allow customers to rate a centre after a confirmed session and show the average rating in search results.
3. **Location-based search:** Store coordinates for each centre, add map views and sort centres by distance from the user.
4. **Scheduled hold cleanup:** Run the expire_holds routine on a schedule so that expired bookings and their payments are updated in the database.
5. **Automated refunds:** Connect cancellations to the payment gateway's refund API with proper merchant configuration.
6. **Notifications:** Send booking confirmations and reminders by email, SMS or WhatsApp.
7. **Wider automated testing:** Add tests for payments, webhooks, cancellation and owner rules, and load-test concurrent bookings on MySQL.
8. **Production deployment:** Deploy behind HTTPS with a production database, static file hosting and monitoring.
9. **Richer owner tools:** Support several centres per owner, staff accounts, recurring session schedules and reports.
10. **Mobile application:** Provide a mobile client that uses the same REST API.

REFERENCES

1. R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.
2. J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Francisco, CA, USA: Morgan Kaufmann, 1993.
3. OWASP Foundation, “OWASP Top 10:2021.” [Online]. Available: https://owasp.org/Top10/ (accessed Oct. 2026).
4. Django Software Foundation, “Django documentation.” [Online]. Available: https://docs.djangoproject.com/ (accessed Oct. 2026).
5. Encode, “Django REST framework.” [Online]. Available: https://www.django-rest-framework.org/ (accessed Oct. 2026).
6. Meta Open Source, “React documentation.” [Online]. Available: https://react.dev/ (accessed Oct. 2026).
7. Razorpay, “Razorpay payments documentation.” [Online]. Available: https://razorpay.com/docs/ (accessed Oct. 2026).
8. N. Leony and G. Prayitno, “Implementation of a web-based online reservation system at Jimmy Salon,” 2025. [Online]. Available: https://pkm.uika-bogor.ac.id/index.php/ABDIDOS/article/view/3110 (accessed Oct. 2026).
9. Y. Adams and R. Tusime, “Design and implementation of a web based reservation system for a hospitality industry: A case study of Paragon Hotel,” Kampala International Univ., School of Computing and Information Technology, Aug. 2012. [Online]. Available: https://ir.kiu.ac.ug/items/b4519619-2403-450e-bef2-78dfed37632e (accessed Oct. 2026).
10. Enestech Software, “SENET: Cloud-based management software for gaming centers.” [Online]. Available: https://senet.cloud/controllers (accessed Oct. 2026).
11. GameBiller, “Gaming cafe management and billing software.” [Online]. Available: https://www.gamebiller.com (accessed Oct. 2026).
12. HandyCafe, “HandyCafe documentation.” [Online]. Available: https://handycafe.com/en/docs/ (accessed Oct. 2026).
13. I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

14. P. P.-S. Chen, “The entity-relationship model—Toward a unified view of data,” ACM Trans. Database Syst., vol. 1, no. 1, pp. 9–36, 1976.

15. Vite, “Vite guide.” [Online]. Available: https://vite.dev/guide/ (accessed Oct. 2026).